DARSHAN PRAJAPATI
MCA-1
ROLL NO: 24
SUBMISSION DATE: 18/03/2026

SHELL SCRIPT PRACTICAL ASSIGNMENT-1

---

1. Write a shell script to display all odd and even numbers using various loops available(for, while and until).

```
echo " Using FOR loop "

for i  in  {1..20}
do
  if [ $((i % 2)) -eq 0 ]
  then
    echo "$i is even "
  else
     echo "$i is odd"
  fi
done

echo
echo "Using While loop "

i=1

while [ $i -le 20 ]
do
   if [ $((i % 2)) -eq 0 ]
       then
         echo "$i is even "
       else
          echo "$i is odd"
       fi
  i=$((i+1))
done
echo

echo "Using UNTIL loop"

i=1
```

```
until [ $i -gt 20 ]
do
  if [ $((i % 2)) -eq 0 ]
    then
       echo "$i is Even"
    else
       echo "$i is Odd"
    fi
    i=$((i+1))
done
```

2. Write a menu driven shell script for basic arithmetic operations.

```
ch=0
while [ $ch -ne 6 ]
do
    echo "1. Addition"
    echo "2. Substraction"
    echo "3. Multiplication"
    echo "4. Division"
    echo "5. Modulus"
    echo "6. Exit"

    echo -n "Enter choice:"
    read ch

    echo -n "Enter no1"
    read  no1
    echo -n "Enter no2"
       read  no2

     if [ $ch -eq  1 ]; then
     ans=$((no1+no2))
     echo "Ans:$ans"

     elif [ $ch -eq 2 ];then
     ans=$((no1-no2))
     echo "Ans:$ans"

     elif [ $ch -eq 3 ];then
        ans=$((no1*no2))
```

```
        echo "Ans:$ans"

    elif [ $ch -eq 4 ];then
        ans=$((no1/no2))
        echo "Ans:$ans"

        elif [ $ch -eq 5 ];then
        ans=$((no1%no2))
        echo "Ans:$ans"

    elif [ $ch -eq 6 ];then
        echo "Bye bye"
    break

    else
    echo "Invalid choice "
    fi
done
```

3. Write a shell script to generate Fibonacci numbers from 1 to n.

```
echo -n "Enter the number:"
read n

a=0
b=1

for ((i=1; i<=n; i++))
do
   echo -n "$b "
   fn=$((a + b))
   a=$b
   b=$fn
done
echo
```

4. write a shell script to generate prime number from 1 to n, where n any positive integer number by user.

```
echo -n "Enter any positive number:"
read n
```

```
echo "Prime Numbers from 1 to $n  are:"

for ((num=2; num<=n; num++))
do
   flag=0
   for((i=2; i<=num/2; i++))
   do
      if [ $((num % i))  -eq 0 ];then
      flag=1
      break
      fi
   done
   if [ $flag -eq 0 ]
    then
   echo -n "$num "
    fi
done
echo
```

5. Script to display patten.

```
|_
| |_
| | |_
| | | |_
| | | | |_
```

```
for((i=1; i<=5; i++))
do
   for((j=1; j<i; j++))
    do
      echo -n "|"
   done
   echo "|_"
done
```

6. Write a menu driven shell script for remove, rename, copy and modify a file

```
ch=0

while [ $ch -ne 5 ]
```

```
do
   echo
   echo "1. Remove a file"
   echo "2. Rename a file"
   echo "3. Copy a file"
   echo "4. Modify(Append) a file"
   echo "5. Exit"
   echo -n "Enter your choice:"
   read ch

   case $ch in
   1)
   echo -n "Enter file name to remove: "
   read file

   if [ -f $file ]
   then
      rm "$file"
      echo "File removed successfully"
   else
      echo "File does not exist."
   fi
   ;;

   2)
   echo -n "Enter old file name: "
      read old
   echo -n "Enter new file name: "
   read new

      if [ -f $old ]
      then
         mv "$old" "$new"
         echo "File renamed successfully"
      else
         echo "File does not exist."
      fi
      ;;

   3)
   echo -n "Enter source file name: "
```

```
    read source
    echo -n "Enter destination file name: "
    read dest

  if [ -f $source ]
  then
    cp "$source" "$dest"
    echo "File copied successfully."
  else
    echo "File does not exist"
  fi
  ;;

  4)

  echo -n "Enter file name to modify: "
       read file

    if [ -f "$file" ]
    then
          echo "Enter text to append:"
          read text
          echo "$text" >> "$file"
          echo "File modified successfully."
     else
          echo "File does not exist."
     fi
       ;;

  5)
    echo "Exiting......."
  ;;

  *)
    echo "Invalid choice"
  ;;
    esac

done
```

7. Write a script to reverse a six-digit number

```
echo -n "Enter the number "
read n

rev=0
while [ $n -gt 0 ]
do
   rem=$((n % 10))
   rev=$((rev * 10 + rem))
   n=$((n / 10))
done
echo "Reverse number is  ${rev}"
```

8. Write a menu driven shell script to find area of rectangle, triangle, and circle

```
ch=0

while [ $ch -ne 4 ]
do
   echo "----------------------------"
   echo "1 : Area of Rectangle"
   echo "2 : Area of Triangle"
   echo "3 : Area of Circle"
   echo "4 : Exit"
   echo "----------------------------"
   echo -n "Enter your choice: "
   read ch

   case $ch in
   1)
   echo -n "Enter length:"
   read l
   echo -n "Enter width:"
   read w
   area=$((l * w))
   echo "Area of Rectangle = $area"
   ;;
    2)
   echo -n "Enter base:"
   read b
```

```
   echo -n "Enter height:"
   read h
   area=$((b * h / 2))
   echo "Area of Triangle = $area"
   ;;
     3)
   echo -n "Enter  radius:"
   read r
   area=$(( 22 * r * r / 7))
   echo "Area of Circle = $area"
   ;;
   4)
   echo "Exiting program..."
   ;;
   *)
   echo "invalid choice!"
   ;;
   esac
done
```

9. Write a script to print message like good morning, good afternoon, …. etc according to the current time.

```
hour=$(date +%H)

if [ $hour -gt 5 ] && [ $hour -lt 12 ]
then
   echo "Good morning"
elif [ $hour -gt 12 ] && [ $hour -lt 17 ]
then
echo "Good Afternoon"

elif [ $hour -gt 17 ] && [ $hour -lt 21 ]
then
echo "Good Evening"

else
   echo "Good night "
fi
```

10. Write a menu driven shell script for storing employee information (like add, modify, delete, display info).

```
ch=0
file="emp.txt"

while [ $ch -ne 5 ]
do
   echo "-----------------------------"
   echo "1 : Add Employee"
   echo "2 : Modify Employee"
   echo "3 : Delete Employee"
   echo "4 : Display Employees"
   echo "5 : Exit"
   echo "-----------------------------"
   echo -n "Enter your choice: "

   read ch
   case $ch in
   1)
   echo -n "Enter Employee id:"
   read id
   echo -n "Enter Employee name:"
   read name
   echo -n "Enter Emplyee Salary:"
   read salary
   echo "$id | $name | $salary ">> $file
   echo "Employee added Succesfully!"
   ;;
   2)
   echo -n "Enter Employee ID to modify: "
         read id
   if grep -q "$id" $file
   then
      grep -v "$id" $file > text.txt
      mv  text.txt $file
      echo "Enter new  details"
      echo -n "Name: "
            read name
            echo -n "Salary: "
            read salary
```

```
      echo "$id | $name | $salary" >> $file
            echo "Employee record modified."
   else
      echo "Employee doesn't exist"
   fi
   ;;
   3)
      echo -n "Enter  Employee Id to delete:"
      read id

      if grep -q "$id" $file
      then
         grep -v "$id" $file > text.txt
         mv text.txt $file
         echo "Employee deleted successfully."
      else
         echo "Employee not found!"
      fi
      ;;
   4)
   if [ -f $file ]
   then
      cat $file
   else
      echo "File not exist"
   fi
   ;;
   5)
   echo "Exiting..."
   ;;
   *)
   echo "Invalid option"
   ;;
   esac
done
```

11. Write a script to print content of the file if file exits otherwise print appropriate message.

```
echo -n "Enter file name:"
read file
```

```
if [ -f "$file" ];then

echo "File contents"
cat $file

else
   echo "file not found"
fi
```

12. Write a script to check the string entered by user is palindrome or not

```
echo -n "Enter the string:"
read str

rev=""

len=${#str}

for ((i=len-1; i>=0; i--))
do
   rev="$rev${str:i:1}"
done

if [ "$str" = "$rev" ]
then
   echo "It is a Palindrome"
else
   echo "Not a Palindrome"
fi
```

13. Write a script to check given number is prime or not.

```
echo -n "Enter the number:"
read n

flag=0
for ((i=2; i<=n/2; i++))
do
   if [ $((n % i)) -eq 0 ]
   then
```

```
      flag=1
      break
   fi
done

if [ $n -le 1 ]
then
   echo "Not a Prime number"

elif [ $flag -eq 0 ]
then
   echo "Prime Number "
else
   echo "Not a Prime number"
fi
```

14. Write a script to display the student marksheet in appropriate format

```
echo "Enter marks of Subject 1:"
read s1

echo "Enter marks of Subject 2:"
read s2

echo "Enter marks of Subject 3:"
read s3

echo "Enter marks of Subject 4:"
read s4

echo "Enter marks of Subject 5:"
read s5

total=$((s1 + s2 + s3 + s4 + s5))
per=$((total /5))

echo "---------------------------------"
echo "       STUDENT MARKSHEET       "
echo "---------------------------------"
echo "Name : $name"
echo "---------------------------------"
```

```
echo "Subject 1 : $s1"
echo "Subject 2 : $s2"
echo "Subject 3 : $s3"
echo "Subject 4 : $s4"
echo "Subject 5 : $s5"
echo "---------------------------------"
echo "Total Marks : $total / 500 "
echo "Percentage  : $per %"
echo "---------------------------------"
```

15. Basic salary of a person is input through the keyboard. His dearness allowance is 40% of basic salary and house rent is 20% of basic salary. Write a program to calculate the gross pay.

```
echo -n "Enter the basic salary:"
read salary

da=$((salary * 40 / 100))
hra=$((salary * 20 / 100))

gpay=$((salary + da + hra))

echo "Basic salary: $salary"
echo "DA: $da"
echo "HRA: $hra"
echo "-------------------"
echo "Gross salary: $gpay"
```

16. The distance between two cities is input through the keyboard. (in km). Write a program to convert this distance into metres, feet, inches and centimetres and display the results.

```
echo -n "Enter distance in kilometers: "
read km

meters=$((km * 1000))
centimeters=$((km * 100000))

feet=$((km * 3281))
inches=$((km * 39370))
```

```
echo "---------------------------------"
echo "Distance in Kilometers : $km km"
echo "Distance in Meters     : $meters m"
echo "Distance in Centimeters: $centimeters cm"
echo "Distance in Feet       : $feet ft"
echo "Distance in Inches     : $inches in"
echo "---------------------------------"
```

17. The script will receive the filename or filename with its full path, the script should obtain information about this file as given by "ls -l" and display it in proper format.e.g., Filename : , File access permissions : , Number of links : , Owner of the file : , Group to which belongs : Size of file : , File modification date : , File modification time : .

```
echo -n "Enter the file name:"
read file

if [ -f "$file" ]
then
   info=$(ls -l $file)
   echo $info

   set -- $info

   perm=$1
   links=$2
   owner=$3
   group=$4
   size=$5
   date=$6
   month=$7
   time=$8
   fname=$9

   echo "--------------------------------------"
   echo "Filename              : $fname"
   echo "File access permissions : $perm"
   echo "Number of links       : $links"
   echo "Owner of the file     : $owner"
   echo "Group of the file     : $group"
   echo "Size of file          : $size bytes"
```

```
   echo "File modification date  : $date $month"
   echo "File modification time  : $time"
   echo "-------------------------------------"
else
   echo "File doesn't exist"
fi
```

18. If cost price and selling price of an item are entered through the keyboard, write a program to determine whether the seller has made profit or loss. Also determine how much profit/loss is made.

```
echo -n "Enter the buy price:"
read bprice
echo -n "Enter the sell price:"
read sprice

net=$((sprice - bprice))

if [ $net -gt 0 ]
then
   echo "Total Profit: $net"

elif [ $net -lt 0 ]
then
   loss=$((bprice - sprice))
   echo "Total Loss: $loss"
else
   echo "No Profit No Loss"
fi
```

19. The script receives two file names as arguments, the script must check whether the files are same or not, if they are similar then delete the second file.

```
f1=$1
f2=$2

if [ ! -f "$f1" ] || [ ! -f "$f2" ];then
   echo "One or Both files do not exist"
   exit
fi
cmp -s "$f1" "$f2"
```

```
if [ $? -eq 0 ];then
   echo "Files are identical."
   rm "$f2"
else
   echo "Files are different"
fi
```

20. Write a shell script to display the menu driven interface :
1) list all files of the current directory 2) print the current directory 3) print the date
4)print the users otherwise display Invalid Option.

```
echo "1) List all files in current directory"
echo "2) Print current directory"
echo "3) Print date"
echo "4) Print users"
echo "--------------------------------"
echo -n "Enter your choice: "
read  ch

case $ch in
1)

   echo "Files in  Currrent directory"
   ls
   ;;

2)
   echo "Current directory"
   pwd
   ;;

3)
   echo "Current Date:"
   date
   ;;

4)
   echo "users"
   who
   ;;
```

```
*)
   echo "invalid option"
   ;;
esac
```

21. Two numbers are entered through the keyboard, find the power, one number raised to another.

```
echo -n "Enter number 1:"
read n1
echo -n "Enter number 2:"
read n2

power=$((n1 ** n2))

echo "Power of $n1 is $power"
```

22. Write a script which has the functionality similar to head and tail commands.

```
echo -n "Enter File name:"
read file

if [ ! -f "$file" ]
then
   echo "File doesnt' exist"
   exit
fi

echo "Display first n lines (head)"
echo "Display last n lines (tail)"
echo -n "Enter your choice"
read ch

echo -n "Enter the lines"
read n

case $ch in
1)
   count=0
   while read line
   do
```

```
      echo "$line"
      count=$((count+1))
      if [ $count -eq  $n ]
      then
         break
      fi
   done < "$file"
   ;;
2)
   total=$(wc -l < "$file")
   start=$((total - n + 1))

   while read line
   do
      count=$((count+1))
      if [ $count -ge  $start ]
      then
         echo "$line"
      fi
   done < "$file"
   ;;
*)
echo "Invalid option"
;;
esac
```

23. The script displays a list of all files in the current directory to which you have read, write and execute permissions.

```
echo "Files with read, write and execute permissions:"

for file in *
do
   if [ -f "$file" ] && [ -r "$file" ] && [ -w "$file" ] && [ -x "$file" ]
   then
   echo "$file"
   fi
done
```

24. The script receives any number of filenames as arguments. It should check whether every argument supplied is a file or directory. If it is a directory, it should be reported. If it is a filename then name of the file as well as the number of lines present in it should be reported.

```
for item in "$@"
do
   if [ -d "$item" ]
   then
      echo "$item is a directory."
   elif [ -f "$item" ]
   then
      lines=$(wc -l < "$item")
      echo "File:$item | Lines: $lines"
   else
      echo "$item is  not a file "
   fi
done
```

25. Display the dates falling on Sundays of the current month.

```
echo "Dates falling on Sundays in the current month:"

cal | awk 'NR>2 {print $1}' | grep -v "^$"
```